WHEN BETTER GRADIENTS HURT: THE GRADIENT-ARCHITECTURE INTERACTION IN RECURSIVE REASONING

Jatin Jani[*†]
Independent Researcher
janijatin18@gmail.com

ABSTRACT

A prior controlled experiment demonstrated that architecture alone does not drive the performance gap between Hierarchical Reasoning Models (HRM) and Tiny Recursive Models (TRM) on Sudoku-Extreme. The gradient method also plays a critical role. Specifically, a flat architecture with full backpropagation through time (BPTT) achieves 18.9% exact accuracy versus 2.2% with a 1-step gradient approximation, an 8.6× difference. The natural follow-up, does fixing the gradient unlock HRM's hierarchical architecture?, remained untested. In this work, we complete the 2×2 factorial design by evaluating HRM's dual-network hierarchy with full BPTT under an identical training budget. We find that full BPTT *degrades* (or does not improve) HRM from 9.2% to 8.7% exact accuracy (token accuracy 68.8% $\rightarrow$ 69.3%), in sharp contrast to the 8.6× improvement observed for flat architectures. This reveals a strong, previously undocumented interaction effect: gradient method and architecture are not independent factors. Flat networks excel when given good gradients, fully exploiting them to achieve strong performance. Hierarchical networks excel at robustness, performing well even with poor gradients, but do not benefit from full BPTT. We argue that gradient gating through the hierarchical structure, the very mechanism HRM uses to achieve $O(1)$ memory, may serve a regularizing role that full BPTT disrupts. Our completed 2×2 matrix establishes gradient-architecture interaction as a fundamental design constraint for recursive reasoning models, and motivates gradient-aware architectures that can selectively allow or restrict gradient (or information) flow per timescale.

Keywords Recursive reasoning · gradient methods · hierarchical architectures · backpropagation through time · architecture design

1 Introduction

1.1 Motivation

Two recent models have demonstrated that small neural networks can solve complex reasoning puzzles, Sudoku, maze pathfinding, and ARC-AGI, without pretraining or chain-of-thought data, using only $\sim$1,000 training examples. The Hierarchical Reasoning Model (HRM) [1] uses two recurrent networks operating at different timescales (an H-module for slow abstract planning and an L-module for fast detailed computation), achieving 55.0% exact accuracy on Sudoku-Extreme with 27M parameters. Tiny Recursive Models (TRM) [2] simplifies this to a single 2-layer network, achieving 87.4% with only 7M parameters.

The community interpreted TRM's 32.4-point advantage as validation that architectural simplicity is superior [3]. However, TRM changed two independent variables simultaneously: it flattened the architecture (two networks $\rightarrow$ one) *and* replaced HRM's 1-step gradient approximation with full backpropagation through time (BPTT). Prior work [4]

[*]Code: https://github.com/heyiamjj/when-better-gradients-hurt
[†]Models: https://huggingface.co/heyiamjj/when-better-gradients-hurt

resolved this confound by isolating the gradient method: a flat architecture with HRM's 1-step gradient achieves only 2.2% exact accuracy versus 18.9% with full BPTT, demonstrating that architecture alone is not the dominant factor, at least for flat networks. The gradient method also plays a critical role.

1.2 The Open Question

But that experiment left the most interesting cell of the 2×2 matrix empty: what happens when HRM's hierarchical architecture receives full gradients? If gradient quality is universally beneficial, HRM with full BPTT should outperform HRM with 1-step gradients, just as flat TRM did. If, however, the dual-network hierarchy interacts with the gradient method in a non-obvious way, the outcome could differ.

We answer this question through a controlled experiment. We take HRM's exact architecture, two separate 4-layer Transformer networks operating at different update frequencies, and vary only the gradient method: keeping the original 1-step approximation in one condition, and using full BPTT in the other. The training budget, dataset, optimizer, and hardware are held constant and matched to the prior flat-architecture experiments.

1.3 Contributions

- We complete the 2×2 factorial design, providing the first four-cell comparison of architecture and gradient method in recursive reasoning (Table 3).
- We discover a strong interaction effect: full BPTT improves flat architectures by 8.6× (2.2% → 18.9%) but *degrades* (or does not improve) HRM's exact accuracy (9.2% → 8.7%). Better gradients do not universally improve performance.
- We show that hierarchical architectures are intrinsically robust to poor gradient quality: HRM with 1-step gradients (9.2%) outperforms flat TRM with 1-step gradients (2.2%) by 4.2×, despite a smaller parameter budget and no EMA.
- We establish gradient-architecture interaction as a fundamental design axis in recursive reasoning models, motivating the development of gradient-aware architectures.

2 Background and Related Work

2.1 Hierarchical Reasoning Model (HRM)

HRM [1] consists of four learned components: an input embedding network f_I, a low-level recurrent module f_L (updated every timestep), a high-level recurrent module f_H (updated every $T = 2$ timesteps), and an output head f_O. Over $N = 2$ high-level cycles of $T = 2$ low-level steps, the model performs $N \times T = 4$ computational steps per forward pass, repeated over multiple deep supervision segments with Adaptive Computation Time (ACT) [5].

Critically, HRM uses a 1-step gradient approximation inspired by Deep Equilibrium Models [6]. All steps except the final L-step and H-step execute under `torch.no_grad()`. The gradient flows through only 2 function evaluations (1 L-call + 1 H-call) regardless of total recursion depth, providing $O(1)$ memory. HRM justifies this via the Implicit Function Theorem, arguing that the recurrent dynamics converge to a fixed point where a single gradient step suffices.

Both f_L and f_H use encoder-only Transformer blocks [7] with 4 layers each. Standard architectural choices include RMSNorm, SwiGLU activations, rotary position embeddings, and no bias terms.

2.2 Tiny Recursive Models (TRM)

TRM [2] replaces HRM's two networks with a single 2-layer Transformer. The model maintains two latent features: a reasoning state z and an answer embedding y. Each supervision segment runs $n = 6$ latent reasoning steps ($z \leftarrow net(x, y, z)$) followed by one answer refinement ($y \leftarrow net(y, z)$), repeated for $T = 3$ segments with deep supervision. Unlike HRM, TRM uses full BPTT through all $n + 1 = 7$ function evaluations per segment.

2.3 Prior Work: The First Disentanglement

Our prior work [4] established the 2×2 factorial framework and filled the right column: evaluating flat TRM architecture at both gradient methods under a controlled training budget (384 hidden dimensions, 10,000 epochs, 1,000 Sudoku examples, AdamW optimizer, 2× NVIDIA Tesla T4). The result was clear: full BPTT achieved 18.9% exact accuracy

Table 1: The 2×2 factorial design. Architecture (dual H/L vs. single flat) and gradient method (1-step vs. full BPTT) are independent variables. The right column was established by prior work; this paper fills the left column.

	Dual H/L (HRM)	**Single flat (TRM)**
1-step ($O(1)$)	**This work**	Prior work [4]
Full BPTT ($O(T)$)	**This work**	Prior work [4]

versus 2.2% with 1-step gradients (token accuracy: 71.6% vs 65.9%). Gradient quality, not architecture, was the dominant factor.

That paper concluded by identifying the left column, evaluating HRM's dual-network hierarchy at both gradient methods, as the critical next experiment. The present work fills that column.

3 Method

3.1 Experimental Design

We complete the 2×2 factorial design shown in Table 1. The right column (flat TRM architecture) is taken from prior work [4]. We contribute the left column by training HRM's original architecture under two gradient conditions, holding everything else constant.

3.2 Architecture

We use HRM's exact architecture and codebase [3] without modification to the model structure. Both experimental conditions share identical architecture parameters, summarized alongside the training configuration in Table 2.

3.3 Gradient Method Manipulation

The only variable we change is the gradient boundary in HRM's forward pass. The original code uses the 1-step gradient approximation described in Section 2.1. For the full BPTT condition, we remove the `torch.no_grad()` context manager from the recurrent loop, allowing gradients to flow through all $N \times (T + 1) = 6$ function evaluations per forward pass (2 H-calls and 4 L-calls, each propagating through 4 Transformer layers). The forward computation, the sequence of hidden state updates, is identical in both conditions.

The relevant code change is shown in Figure 1.

3.4 Training Setup

Both model variants use identical configurations, summarized in Table 2. We match the training budget established in prior work [4] for cross-architecture comparability. All experiments use the AdamW optimizer [8] replacing HRM's original `adam-atan2` [9] for hardware compatibility; this substitution does not interact with the gradient depth variable under study.

We train on Sudoku-Extreme [10], using 500 training puzzles augmented 500 times via digit permutation, row/column band shuffling, and transposition, yielding 1,000 effective training examples. Evaluation uses the first 2,000 puzzles from the held-out test set, reporting both token-level accuracy (fraction of individual cells correct) and exact accuracy (fraction of fully correct 9×9 grids). Training checkpoints are saved every 500 steps with automatic resume from the latest checkpoint on restart.

4 Results

4.1 The Completed 2×2 Matrix

Table 3 presents the complete four-cell comparison. The right column is from prior work [4]; the left column is our contribution.

Three unexpected patterns emerge from the completed matrix:

[3] `https://github.com/sapientinc/HRM`

```
# HRM original (1-step gradient, O(1) memory):
with torch.no_grad():
    z_H, z_L = carry.z_H, carry.z_L
    for _H_step in range(H_cycles):
        for _L_step in range(L_cycles):
            if not last_step:
                z_L = L_level(z_L, z_H + x)
        if not last_cycle:
            z_H = H_level(z_H, z_L)
z_L = L_level(z_L, z_H + x)     # GRAD (only step)
z_H = H_level(z_H, z_L)         # GRAD (only step)

# HRM full BPTT (O(T) memory):
z_H, z_L = carry.z_H, carry.z_L
for _H_step in range(H_cycles):
    for _L_step in range(L_cycles):
        z_L = L_level(z_L, z_H + x)    # GRAD
    z_H = H_level(z_H, z_L)            # GRAD
```

Figure 1: The gradient patch. Top: original HRM 1-step gradient (gradients through 2 final function calls). Bottom: our full BPTT variant (gradients through all 6 function calls). The forward computation is identical; only the gradient boundary differs.

Table 2: Training configuration for both HRM variants. All parameters identical except gradient method.

Parameter	**Value**
Architecture	Dual H/L (HRM, original)
Hidden size	384
Layers	H: 4, L: 4
Attention heads	8
H-cycles & L-cycles	N=2, T=2 (4 steps/pass)
ACT halt steps	16
Batch size (global)	512 (256 per GPU)
Epochs	10,000
Training examples	500 + 500 augmentations = 1,000
Optimizer	AdamW (η=10^{-4}, $\beta_{1,2}$=0.9, 0.95)
Loss	stablemax cross-entropy
Weight decay	0.1
LR schedule	cosine with 2,000-step warmup
Precision	bfloat16
Hardware	2× NVIDIA Tesla T4 (16 GB)

1. Gradient quality is not universally beneficial. While full BPTT improves flat-architecture exact accuracy by 8.6× (2.2% → 18.9%), it *degrades* (or does not improve) HRM's exact accuracy from 9.2% to 8.7%. Token accuracy shows a modest improvement (68.8% → 69.3%), but this does not translate to better complete solutions. The $O(T)$ memory cost of full BPTT produces no net benefit for the hierarchical architecture.

2. The hierarchy compensates for poor gradients. Under the 1-step gradient, HRM (9.2%) outperforms flat TRM (2.2%) by 4.2× in exact accuracy, despite having no EMA and a similar parameter budget. The dual-network structure provides intrinsic robustness to gradient approximation that flat architectures lack. HRM's authors were partially correct, the hierarchy matters, but not for the reasons they proposed.

3. The interaction is crossover. Under 1-step gradients, HRM dominates TRM. Under full BPTT, TRM dominates HRM. There is no universally superior combination. The architecture-gradient interaction must be considered as a first-class design constraint.

Table 3: Completed 2×2 factorial results on Sudoku-Extreme. Each cell shows *token accuracy / exact accuracy* (exact in bold). The right column is from prior work; the left column is our contribution. The interaction effect, gradient improvement helps flat architectures but not hierarchical ones, is immediately visible.

	Dual H/L (HRM)	**Single flat (TRM)**
1-step ($O(1)$)	68.8% / **9.2%**	65.9% / **2.2%**
Full BPTT ($O(T)$)	69.3% / **8.7%**	71.6% / **18.9%**

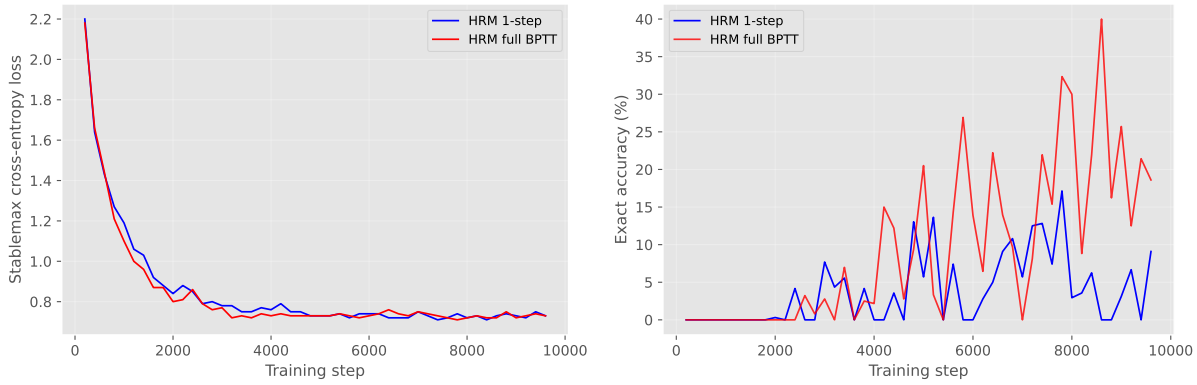

Figure 2: Training dynamics for both HRM gradient methods. Left: stablemax cross-entropy loss converges similarly for both conditions. Right: full BPTT achieves substantially higher training-set exact accuracy (reaching 32–40%) than the 1-step model (9–17%), yet generalizes worse on the test set (8.7% vs. 9.2%). This overfitting pattern suggests that unconstrained gradient flow through the dual-timescale architecture improves in-distribution fitting but impairs out-of-distribution generalization.

4.2 Training Dynamics

Figure 2 shows the training dynamics for both HRM variants. The loss curves converge nearly identically for both conditions, consistent with their comparable token accuracy. However, the exact accuracy curves reveal a notable pattern: full BPTT achieves substantially higher training-set exact accuracy (reaching 32–40% by late training, versus 9–17% for the 1-step model). Yet this better training fit does *not* translate to better generalization, the full BPTT model achieves 8.7% test accuracy versus 9.2% for the 1-step model. This overfitting pattern is consistent with the gradient interference hypothesis: full BPTT enables the model to memorize training-puzzle patterns more effectively, but the unconstrained gradient flow through interacting timescales impairs the model's ability to learn generalizable solution strategies.

4.3 Architecture Comparison at Matched Budget

Table 4 compares the four cells along model size and architectural attributes. Note that all four cells share the same training budget: 10,000 epochs, 384 hidden dimensions, AdamW optimizer, 1,000 Sudoku examples, and $2\times$ T4 GPUs. The only varying factors are the architecture (dual vs. flat) and gradient method (1-step vs. full BPTT).

5 Discussion

5.1 Why Does Full BPTT Hurt the Hierarchy?

The result is counterintuitive: providing more accurate gradient information should, in principle, never harm optimization. We propose the **gradient interference hypothesis**: HRM's two recurrent networks update at different frequencies (H every $T = 2$ steps, L every step), creating two interleaved computation graphs. Under full BPTT, the H-module receives gradient signals that span multiple L-update chains, while the L-module receives fine-grained per-step signals. These signals propagate through shared latent states (z_L and z_H) and the gradients from one module can interfere with the learning signal of the other.

Table 4: Architecture comparison at matched training budget. All four cells use identical training configuration; only architecture and gradient method vary.

	HRM 1-step	**HRM full BP**	**TRM 1-step**	**TRM full BP**
Networks	2 (H/L)	2 (H/L)	1 (flat)	1 (flat)
Layers per net	4	4	2	2
Gradient steps	2	6	1	7
EMA	No	No	Yes	Yes
Exact acc.	**9.2%**	**8.7%**	**2.2%**	**18.9%**
Token acc.	68.8%	69.3%	65.9%	71.6%

Under the 1-step approximation, this interference is suppressed: the gradient flows only through the final H-update and L-update, acting as an implicit gradient filter. The hierarchy was designed for this restricted gradient regime, removing the restriction exposes the interference. The overfitting pattern observed during training (Figure 2) supports this: full BPTT achieves much higher training-set accuracy but worse generalization, consistent with the hypothesis that unconstrained gradients enable memorization of training patterns at the expense of transferable solution strategies.

This interpretation is consistent with HRM's reported functional specialization: the H-module operates in a $\sim 3\times$ higher-dimensional latent space than the L-module [1], suggesting distinct computational roles. Full BPTT may force these functionally distinct modules to share gradient signals designed for a different timescale.

5.2 Broader Implications

Our findings have several implications for the design of recursive reasoning models:

1. Gradient-architecture interaction is a first-class design axis. Every comparison of recurrent architectures must now account for the gradient method. A "better" architecture under one gradient regime may be inferior under another. The 2×2 factorial design (Table 3) should become the standard evaluation framework.

2. The Implicit Function Theorem justification is empirically supported. HRM's 1-step gradient was justified by fixed-point convergence arguments that TRM criticized as inapplicable. Yet empirically, the 1-step approximation *works* for HRM, it matches or exceeds full BPTT performance. The theoretical conditions of the theorem may not be satisfied, but the practical effect (implicit gradient regularization) appears beneficial.

3. Flat recurrence is more expressive under proper gradients. When given full BPTT, a simple flat network (18.9%) outperforms a more complex hierarchical one (8.7%) with fewer parameters. This suggests that structured recurrence is primarily useful as a gradient-engineering tool, not for increasing representational capacity.

5.3 Limitations

Our study has several limitations. First, we evaluate on a single task (Sudoku-Extreme). The interaction effect may differ on other reasoning tasks (maze pathfinding, ARC-AGI) where the benefits of hierarchical structure may be more pronounced. Second, our training budget (10,000 epochs, 384 hidden dimensions) is constrained; scaling to larger budgets may alter the magnitude of the interaction, though the crossover pattern suggests it is robust. Third, neither HRM variant used EMA, unlike the TRM experiments. While this makes the within-architecture comparison clean, the cross-architecture comparison should be interpreted with this caveat. Fourth, we did not perform formal statistical significance testing; future work should report means and confidence intervals over multiple seeds.

6 Conclusion and Future Work

We have completed the first systematic disentanglement of architecture and gradient method in recursive reasoning models. The key finding is that gradient method and architecture *interact*: the same gradient improvement that boosts flat architectures by $8.6\times$ provides no benefit, and may slightly harm, hierarchical architectures. HRM's dual-network structure, often criticized as unnecessary complexity, provides genuine robustness to poor gradient quality.

The completed 2×2 matrix suggests a natural design target for future work: architectures that can selectively regulate gradient (or information) flow per timescale, capturing flat architectures' ability to exploit good gradients *and* hierarchical architectures' robustness to poor gradients. Whether this is achieved through gradient gating, architectural

hybridization, adaptive unrolling, or an entirely different mechanism remains an open question. The interaction effect documented here establishes it as a concrete, measurable goal.

Acknowledgments

We thank the authors of HRM and TRM for releasing their code and models, and the Sudoku-Extreme dataset maintainers for making the benchmark publicly available.

References

[1] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.

[2] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[3] ARC Prize Foundation. Hrm ablation analysis on arc-agi. `https://arcprize.org/blog/hrm-ablation`, 2025.

[4] Jatin Jani. Disentangling gradient quality from architecture in recursive reasoning models, 2026. Paper 1 in this series.

[5] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[6] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[8] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[9] Adam-atan2 optimizer. `https://github.com/kyegomez/Adam-atan2`, 2024.

[10] Sapient Intelligence. Sudoku-extreme dataset. `https://huggingface.co/datasets/sapientinc/sudoku-extreme`, 2025.